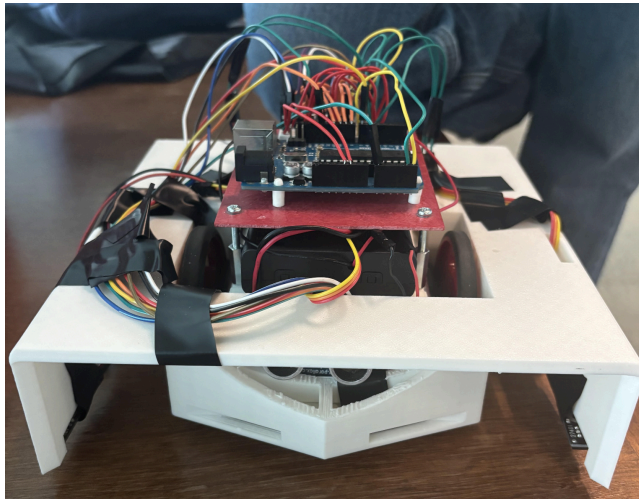


MAE3780 Final Report

Team Number: 51

Team Members: [Rachel Landa, rsl85], [Aaron Chin, adc263]

1. Robot Design and Strategy Overview



Our robot was designed for the MAE 3780 robotics competition with a focus on board control and physical dominance. Built on the standard BOE-Bot platform, its primary mechanical feature was an 8 in. × 8 in. outer shell surrounding the chassis (see Appendix C). The shell protected the wheels from cube jamming while creating internal space for passive cube collection. A slanted front edge guided cubes into the side openings as the robot drove forward. The robot was kept low to the ground for stability and pushing effectiveness, with servos mounted inward to maximize internal collection space.

The robot used two QTI sensors for black border detection and the standard continuous-rotation servo drives. Ultrasonic and 2 color sensors were wired but unused in the competition code main to lessen complexity (See Appendix B for wiring diagram). The Ultrasonic sensor were disconnected before the competition, so they were only wired during testing.

The software strategy was deliberately simple and robust: drive forward after a short random initial turn, continue until a QTI sensor detected the border, then briefly reverse, perform a timed turn, and resume forward motion. The overall behavior leveraged physical reliability over complex sensing or targeting (Appendix D, Appendix E). It limited system errors to reliably sense the playing field edge and resume passive collection.

2. Design Process Reflection

The design process began by identifying key failure modes: wheel obstruction, weak pushing power, and driving off the board. This led us to prioritize mechanical protection, a wide, stable footprint, and reliable border detection over active or complex systems.

At first, we chose a passive cube collection using a cardboard structure and one color sensor. We had a difficult time reliably sensing the edge of the playing field with this setup and needed the sensor to be further in front of the robot to sense color accurately before driving off the board. We then swapped to a wide rigid chassis to securely attach sensors ~2in. in front of the drive wheels to ensure we wouldn't drive off the playing field. This was a major issue we had during milestone 4, and we fixed it accordingly for the competition.

Another issue we had was COM (center-of-mass) location. In milestone 4, we had an issue with inconsistent turning because of the COM being located too far back on the robot and not enough weight on the drive wheels. During prototyping of the main chassis we had for competition, we made sure to create a wide and flat structure with a low COM and ensure the COM was very close to the drive wheels for reliable control.

A large portion of time was spent on testing drive timing. We had to calibrate each servo with a PWM offset to ensure our robot could drive straight, backwards, and turn reliably. We tackled some of this issue in milestone 2, but it became more apparent when we wanted the robot to drive across the whole playing board accurately. We were looking to have different drive speeds, but this proved difficult because of a changing offset depending on the desired speed. For example, driving at a fast speed needed a PWM offset of about 28 on one wheel, but at a slow speed we needed an offset of 20. This variability would have taken extensive testing to find correct offsets for each speed, so we settled for 1 speed for the whole competition.

One Software development focused on a fast QTI sampling loop (Appendix E, under QTI) with tuned thresholds and timed turns for consistent border avoidance. A major roadblock was cube retention; cubes often slid out during turns or collisions. Ultimately, the design's greatest strengths were its mechanical simplicity, wheel protection, and strong physical presence, while the main weakness remained the lack of secure cube trapping.

3. Competition Analysis

Our robot finished the tournament with a 3-3-1 record. It performed well in direct physical interactions due to its low COM design. The robot excelled at pushing opponents, surviving collisions, and keeping its wheels clear thanks to the outer shell. The large contact area also helped control space on the board.

The QTI border detection strategy worked reliably in most matches. In competition, it successfully detected the border every time it was supposed to and had no errors. Upon detecting the edge, the robot typically reversed briefly, turned, and continued driving, staying active without driver intervention. Its simple forward-driving approach ensured it was almost always engaged, be that pushing cubes, fighting opponents, or

repositioning. This strategy worked well, as we would collide with opponents and push them until the border was detected and the opponent was therefore off the playing field.

The main weakness was cube retention. While cubes easily entered the shell, they frequently slid out during maneuvers or impacts, leading to inconsistent scoring. The lack of active targeting was also a double-edged sword: it reduced complexity and failure points but prevented intentional strategy. Overall, the robot's mechanical robustness and reliable border avoidance were its greatest strengths on competition day.

4. Conclusions

The robot proved to be a successful, simple, and durable competitor. Its large protective shell, low chassis, and straightforward QTI-based control loop enabled consistent performance and a competitive 3-3-1 record, particularly in pushing interactions.

If we could redo the project under the same constraints, we would add one-way flaps or flexible hinges to the shell openings to improve cube retention. We would retain the low, wide chassis but refine the post-border turning routine for greater consistency across varying battery and surface conditions.

One other issue we would address in the future is the cost of our 3D printed Chassis. This chassis was the main component of our robot, but was severely over budget. We had limited time to fix this before competition because of time constraints. We propose possible solutions for this in Appendix A, some of these being weight reduction or material change.

Advice for next year's students: Prioritize reliability and simplicity over complexity. A robot that keeps moving, protects its wheels, and stays on the board will usually outperform more ambitious but fragile designs. Test extensively under realistic conditions with collisions and cubes. Most importantly, focus not only on collecting cubes but on keeping them inside the robot.

Appendix A: Bill of Materials

Item	Quantity	Purpose	Cost	Used in Final Robot?
Standard BOE-Bot Kit Components	1 kit	Base chassis, Arduino/control board, servos, wiring	Provided	Yes
QTI Sensors	2	Detect black border	Provided + 1 additional free	Yes
Free components (not in kit) - Wires, standoffs/screws, 100kOhm resistors	N/A	Wiring, assembly, and lift circuitry to create space for batteries. Resistors for QTI	Provided	Yes
Custom PLA Outer Shell/Casing ~237.61g using RPL pricing	1	Protect wheels and create cube-collection space	\$96.05	Yes
Fasteners/Tape/Adhesive	1	Attach shell and components	\$6	No
Extra Color Sensor	1	Detect color on corners of robot instead of directly in front	\$5	No
Ultrasonic Sensor	1	Opponent detection	Provided	No

Total costs: \$11 + 3D-printed component (\$96.05) = **\$107.05**

Our main cost was our 3D printed Chassis which was way over budget. Solutions for reducing total costs to under \$40. Aim for outer casing to be ~\$30.

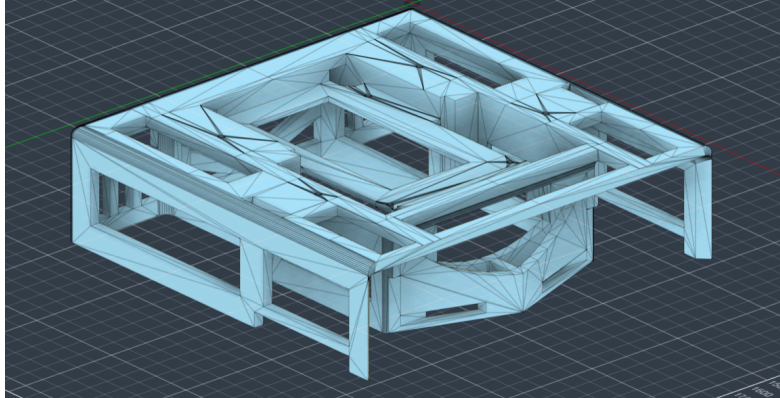
1. Equivalent casing, with more cutouts: This design was made under tight time constraints and can be simplified to main structural components (columns, cross beams, and supporting beams). The wall thickness was also $\frac{1}{8}$ in. which can also be reduced to a still workable threshold of about $\frac{1}{8}$ in which is a reasonable 3mm.
 - a. Given the reduction of approx. 5mm to 3mm this is about 40% reduction
 - b. Conceptual cutouts (done quickly and can likely be reduced further) reduced the original 237.61g to 177.39g

c.

Filament	Model
■ 1	58.53 m 177.39 g

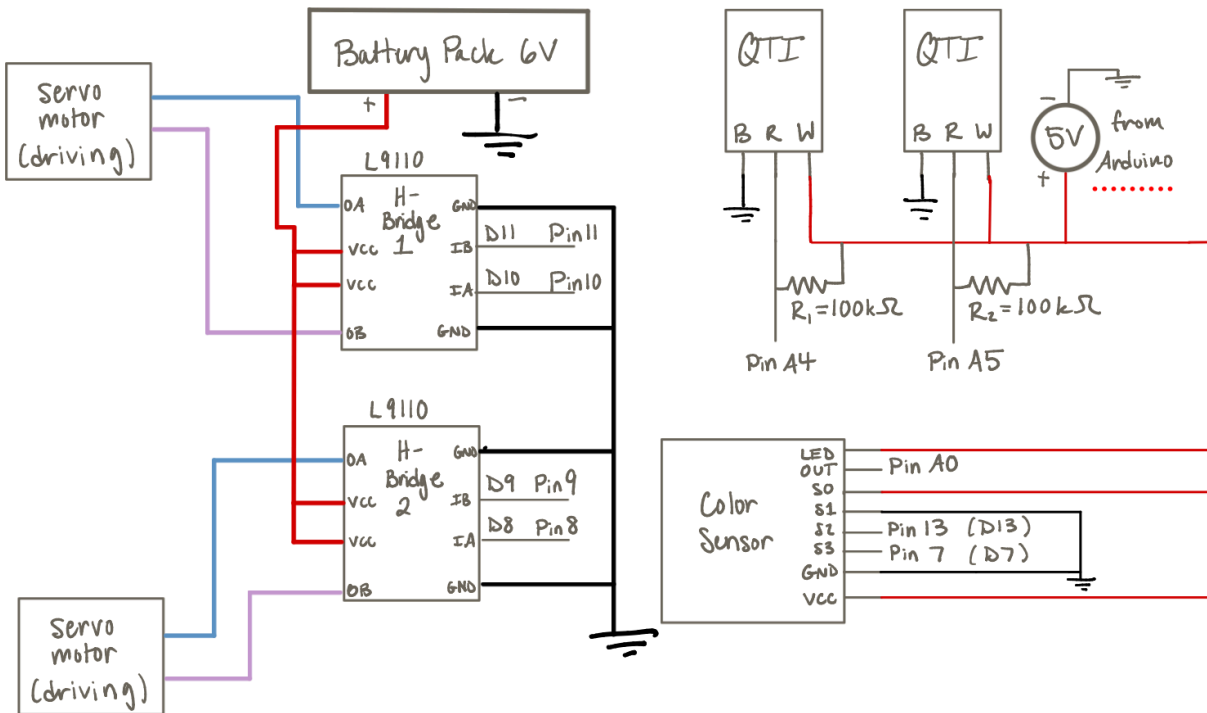
- d. Given 40% reduction from wall thickness and 177.39g from just conceptual cutouts, we can show an easily achievable reduction to 106.43g. This correlated to \$43 from RPL pricing.
- e. From quick modifications (eyeballing cuts) we were able to conceptualize a new chassis that is much more near the budget.
- f. We can deduce a Chassis cost of \$30 is very much possible with a reasonable 3mm wall thickness, and well planned beam usage.

Image of conceptual cutouts to reduce material used

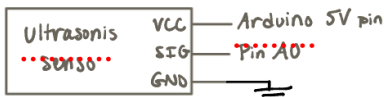


- 2. Laser cutting + later assembly rather than 3D printing:
 - a. Laser cutting material is significantly cheaper than 3D printing through the RPL.
 - b. Since our design is boxy and uses large panels, this is easily translatable to a laser cut design. Laser cut components can be screwed together or even superglued which should hold enough for collisions against other robots
- 3. Cardboard recreation:
 - a. Because the model is boxy, it can also be recreated with cardboard given large panels, and cutting out multiple layers for supporting the structure.
 - b. Some drawbacks for this method is support cardboard inside the main structure reduces internal volume to collect cubes.
 - c. A better solution would be to combine 3D printed material with super glued cardboard support beams. This can give an okay starting frame that is sturdier than cardboard by itself, but will have more support than just the 3D printed material with many cutouts.

Appendix B: Circuit Diagram



Ultrasonic sensor not wired during competition, but this is the wiring during testing

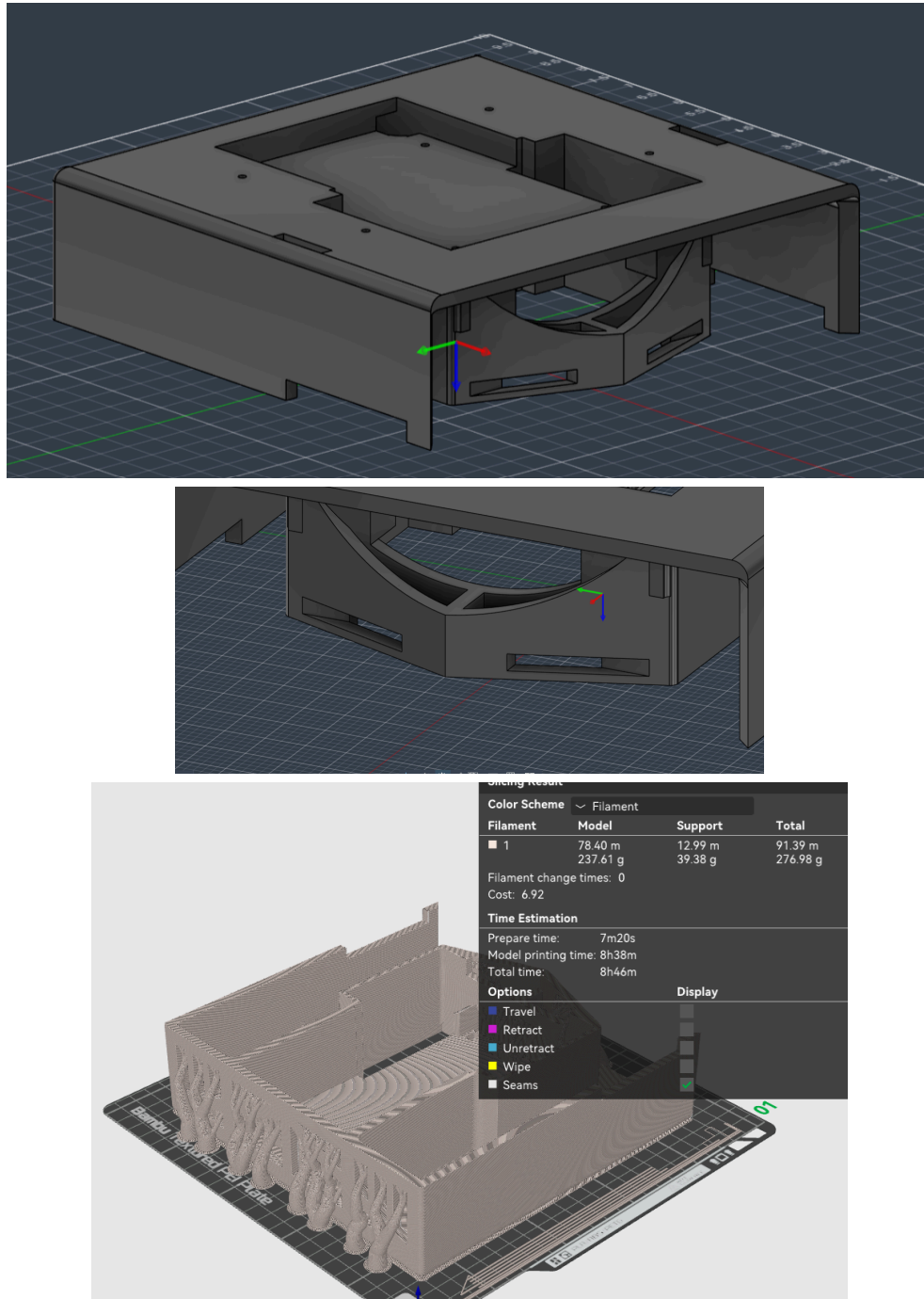


The final robot used the standard BOE-Bot motor setup with two continuous rotation servos and two QTI sensors for border detection. The QTI sensors and servos were connected as follows:

Component	Signal	Arduino Pin
Left QTI sensor	Analog output	A5
Right QTI sensor	Analog output	A4
Left motor direction/control	Motor control	D8/D9
Right motor direction/control	Motor control	D10/D11

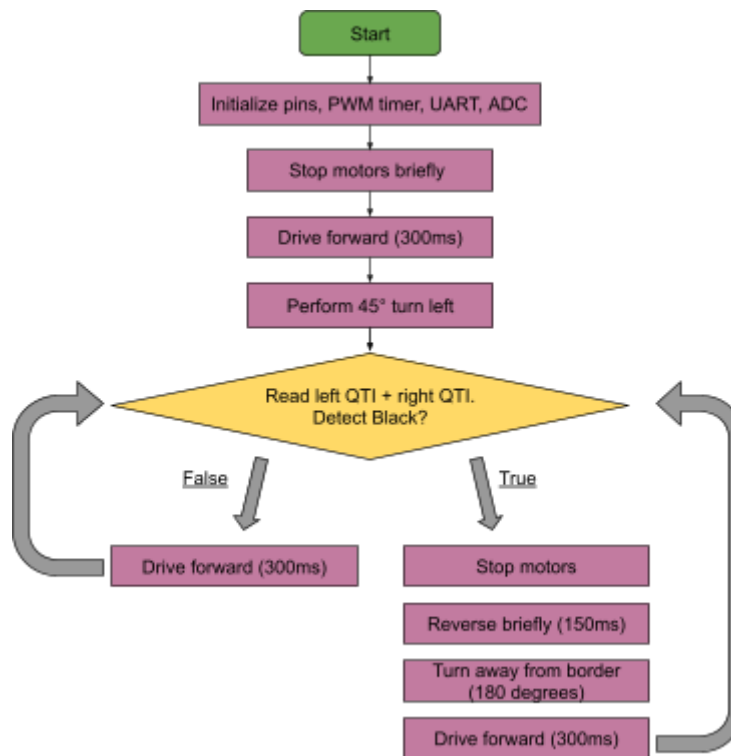
The Arduino code also included pin definitions for ultrasonic and color sensors, but the final competition loop mainly used the QTI sensors and motor outputs. During competition we kept the color sensor wired, but unplugged the ultrasonic sensor.

Appendix C: CAD Files and Drawings



The most important manufactured part was the large outer shell, approximately 8 in. by 8 in. This shell surrounded the robot and protected the wheels from cube interference. The slanted front guide was designed to redirect cubes toward the side openings of the shell. The servos were positioned closer to the inside of the robot so that the shell could fit around the chassis while leaving space for cubes to enter the casing. Weight from BambuLabs: 237.61g

Appendix D: Flowchart



This is the flow chart for our strategy logic which we used in our Arduino Code. We follow a simple initialization procedure, then constantly check for the border. When the border is detected, the robot turns around and continues on. This repeats.

Appendix E: Code

The final Arduino code initializes the robot's pins, PWM outputs, UART communication, ADC readings, and motor control functions. The final competition loop uses the two QTI sensors to detect the black border. When black is detected, the robot stops, backs up, turns away from the border, and resumes driving forward. The code also includes functions for ultrasonic and color sensing, but these were not central to the final competition behavior.

```
// Robot w/ QTI sensors, ultrasonic sensor, color sensors, and motor control
```

```
// Ultrasonic: D2
```

```
// left QTI = A5; right QTI = A4
```

```
// left color sensor:
```

```
// S2 = D5; S3 = D6; OUT = A1
```

```
// right color sensor:
```

```
// S2 = D13; S3 = D7; OUT = A0
```

```
// Motor driver:
```

```
// left DIR = D8; left PWM = D9; right PWM = D10; right DIR = D11
```

```
#include <avr/io.h>
```

```
// Color IDs
```

```
#define COLOR_BLUE 0
```

```
#define COLOR_YELLOW 1
```

```
#define COLOR_BLACK 2
```

```
// Motor speeds
```

```
// lower PWM = faster
```

```
#define FORWARD_SPEED 24
```

```
#define DRIVE_SPEED 200
```

```
#define EASE_FAST 40
```

```
#define EASE_SLOW 90
```

```
#define TURN_SPEED 40
```

```
// Turn timing
```

```

#define TURN_90_TIME_MS 400

#define TURN_180_TIME_MS 950


// Ultrasonic threshold

#define OBJECT_THRESHOLD_COUNT 2500


// QTI thresholds

#define left_QTI_BLACK_THRESHOLD 1023

#define right_QTI_BLACK_THRESHOLD 400


// Random seed

unsigned long random_state = 12345;


// UART setup

void uart_init() {

    UBRR0H = 0;

    UBRR0L = 103;

    // enable transmitter

    UCSR0B = 0b00001000;

    // 8-bit data

    UCSR0C = 0b00000110;

}


// Send one character

void uart_send_char(char c) {

    while (!(UCSR0A & 0b00100000));

    UDR0 = c;

}


// Send full string

void uart_send_string(const char* str) {

```

```

    while (*str != '\0') {

        uart_send_char(*str);

        str++;

    }

}

// Send number through UART

void uart_send_number(long num) {

    char buffer[12];

    int i = 0;

    if (num == 0) {

        uart_send_char('0');

        return;

    }

    if (num < 0) {

        uart_send_char('-');

        num = -num;

    }

    // store digits backwards

    while (num > 0) {

        buffer[i] = (num % 10) + '0';

        num /= 10;

        i++;

    }

    // print digits in correct order

    while (i > 0) {

        i--;

        uart_send_char(buffer[i]);

    }

}

```

```

// Approx ms delay

void delay_ms(int ms) {

    for (int i = 0; i < ms; i++) {

        for (volatile int j = 0; j < 1600; j++) {

            __asm__("nop");

        }

    }

}

```

```

// Approx us delay

void delay_us(int us) {

    for (int i = 0; i < us; i++) {

        for (volatile int j = 0; j < 16; j++) {

            __asm__("nop");

        }

    }

}

```

```

// ADC setup

void adc_init() {

    // AVcc reference

    ADMUX = 0b01000000;

    // enable ADC, prescaler 128

    ADCSRA = 0b10000111;

}

```

```

// Read ADC channel

int adc_read(unsigned char channel) {

    // select channel

    ADMUX = 0b01000000 | (channel & 0b00001111);

    // start conversion

```

```

    ADCSRA |= 0b01000000;

    // wait until done

    while (ADCSRA & 0b01000000);

    return ADC;
}

```

```

// Pin setup

void pins_init() {

    // D5,D6,D7 outputs

    DDRD = 0b11100000;

    // no pullup on D2

    PORTD &= 0b11111011;

    // motor pins + D13 output

    DDRB = 0b00101111;

    // PORTC inputs

    DDRC = 0b00000000;

}

```

```

// PWM setup

void timer1_pwm_init() {

    // Fast PWM

    TCCR1A = 0b10100001;

    TCCR1B = 0b00001011;

    // motors stopped

    OCR1A = 255;

    OCR1B = 255;

}

```

```

// Left motor forward

void set_left_motor_forward() {

    PORTB |= 0b00000001;

}

```

```

// Left motor backward

void set_left_motor_backward() {

    PORTB &= 0b11111110;

}

// Right motor forward

void set_right_motor_forward() {

    PORTB |= 0b00001000;

}

// Right motor backward

void set_right_motor_backward() {

    PORTB &= 0b11110111;

}

// Set motor speeds

void set_motor_speeds(unsigned char left_speed,unsigned char right_speed) {

    OCR1A = left_speed;

    OCR1B = right_speed;

}


// Move forward

void move_forward() {

    set_left_motor_forward();

    set_right_motor_forward();

    // small correction for drift

    set_motor_speeds(FORWARD_SPEED,FORWARD_SPEED + 24);

}


// Move backward

void move_backward() {

    set_left_motor_backward();

    set_right_motor_backward();

    set_motor_speeds(DRIVE_SPEED + 24,DRIVE_SPEED);

```

```
}
```

```
// Stop motors
```

```
void stop_motors() {  
  
    set_left_motor_forward();  
  
    set_right_motor_forward();  
  
    // 255 = stop  
  
    set_motor_speeds(255,255);  
  
}
```

```
// Turn right ~90 deg
```

```
void turn_right_90() {  
  
    stop_motors();  
  
    delay_ms(100);  
  
  
    set_left_motor_forward();  
  
    set_right_motor_backward();  
  
    set_motor_speeds(TURN_SPEED,TURN_SPEED);  
  
    delay_ms(TURN_90_TIME_MS);  
  
    stop_motors();  
  
    delay_ms(100);  
  
}
```

```
// Turn left ~90 deg
```

```
void turn_left_90() {  
  
    stop_motors();  
  
    delay_ms(100);  
  
    set_left_motor_backward();  
  
    set_right_motor_forward();  
  
    set_motor_speeds(TURN_SPEED,TURN_SPEED);  
  
    delay_ms(TURN_90_TIME_MS);  
  
}
```

```

        stop_motors();

        delay_ms(100);
    }

// Turn around
void turn_180() {

    stop_motors();

    set_left_motor_forward();

    set_right_motor_backward();

    set_motor_speeds(TURN_SPEED, TURN_SPEED);

    delay_ms(TURN_180_TIME_MS);

    stop_motors();
}

// Read left QTI
int read_left_qti_raw() {

    return adc_read(5);
}

// Read right QTI
int read_right_qti_raw() {

    return adc_read(4);
}

// Check if left QTI sees black
int left_qti_black() {

    int value = read_left_qti_raw();

    if (value >= left_QTI_BLACK_THRESHOLD) {

        return 1;

    } else {

        return 0;
    }
}

```



```

    }

}

// Check if right QTI sees black

int right_qti_black() {
    int value = read_right_qti_raw();

    if (value >= right_QTI_BLACK_THRESHOLD) {
        return 1;
    } else {
        return 0;
    }
}

// Print QTI readings

void print_qti_values() {
    int left_value = read_left_qti_raw();
    int right_value = read_right_qti_raw();

    uart_send_string("left QTI A5=");
    uart_send_number(left_value);

    if (left_value >= left_QTI_BLACK_THRESHOLD) {
        uart_send_string(" BLACK");
    } else {
        uart_send_string(" NOT_BLACK");
    }

    uart_send_string(" right QTI A4=");
    uart_send_number(right_value);

    if (right_value >= right_QTI_BLACK_THRESHOLD) {
        uart_send_string(" BLACK");
    } else {
        uart_send_string(" NOT_BLACK");
    }
}

```

```

        uart_send_string("\r\n");
    }

// Ultrasonic pin output
void ultrasonic_pin_output() {
    DDRD |= 0b00000100;
}

// Ultrasonic pin input
void ultrasonic_pin_input() {
    DDRD &= 0b11111011;
    PORTD &= 0b11111011;
}

// Ultrasonic HIGH
void ultrasonic_pin_high() {
    PORTD |= 0b00000100;
}

// Ultrasonic LOW
void ultrasonic_pin_low() {
    PORTD &= 0b11111011;
}

// Read ultrasonic pin
int read_ultrasonic_pin() {
    if (PIND & 0b00000100) {
        return 1;
    } else {
        return 0;
    }
}

```

```
}
```

```
// Read ultrasonic pulse
```

```
long read_ultrasonic_raw() {
```

```
    long count = 0;
```

```
    long timeout = 0;
```

```
    ultrasonic_pin_output();
```

```
    ultrasonic_pin_low();
```

```
    delay_us(2);
```

```
    // trigger pulse
```

```
    ultrasonic_pin_high();
```

```
    delay_us(10);
```

```
    ultrasonic_pin_low();
```

```
    ultrasonic_pin_input();
```

```
    // wait for echo
```

```
    timeout = 0;
```

```
    while (read_ultrasonic_pin() == 0) {
```

```
        timeout++;
```

```
        if (timeout > 30000) {
```

```
            return -1;
```

```
        }
```

```
    }
```

```
    // measure pulse
```

```
    count = 0;
```

```
    while (read_ultrasonic_pin() == 1) {
```

```
        count++;
```

```
        if (count > 30000) {
```

```
            return -2;
```

```

    }

}

return count;
}

// Check if object detected

int check_object() {

    long raw = read_ultrasonic_raw();

    if (raw == -1 || raw == -2) {

        return 0;

    }

    // smaller count = closer object

    if (raw < OBJECT_THRESHOLD_COUNT) {

        return 1;

    } else {

        return 0;

    }

}

// Left sensor RED filter

void left_select_red() {

    PORTD &= 0b10011111;

}

// Left sensor GREEN filter

void left_select_green() {

    PORTD |= 0b01100000;

}

// Left sensor BLUE filter

```

```

void left_select_blue() {

    PORTD &= 0b11011111;

    PORTD |= 0b01000000;

}

// Right sensor RED filter

void right_select_red() {

    PORTB &= 0b11011111;

    PORTD &= 0b01111111;

}

// Right sensor GREEN filter

void right_select_green() {

    PORTB |= 0b00100000;

    PORTD |= 0b10000000;

}

// Right sensor BLUE filter

void right_select_blue() {

    PORTB &= 0b11011111;

    PORTD |= 0b10000000;

}

// Read left color OUT

int read_left_color_out() {

    if (PINC & 0b00000010) {

        return 1;

    } else {

        return 0;

    }

}

```

```

// Read right color OUT

int read_right_color_out() {

    if (PINC & 0b00000001) {

        return 1;

    } else {

        return 0;

    }

}

// Read left color pulse

long read_left_color_pulse() {

    long count = 0;

    long timeout = 0;

    while (read_left_color_out() == 1) {

        timeout++;

        if (timeout > 30000) {

            return 30000;

        }

    }

    while (read_left_color_out() == 0) {

        count++;

        if (count > 30000) {

            return 30000;

        }

    }

    return count;

}

// Read right color pulse

long read_right_color_pulse() {

```

```
    long count = 0;

    long timeout = 0;

    while (read_right_color_out() == 1) {

        timeout++;

        if (timeout > 30000) {

            return 30000;

        }

    }

    while (read_right_color_out() == 0) {

        count++;

        if (count > 30000) {

            return 30000;

        }

    }

    return count;

}
```

```
// Average left color reading

long read_left_color_avg(int samples) {

    long total = 0;

    for (int i = 0; i < samples; i++) {

        total += read_left_color_pulse();

    }

    return total / samples;

}
```

```
// Average right color reading

long read_right_color_avg(int samples) {

    long total = 0;

    for (int i = 0; i < samples; i++) {

        total += read_right_color_pulse();

    }

}
```

```

    }

    return total / samples;
}

// Read left RGB
void read_left_rgb(long* r,long* g,long* b) {

    left_select_red();

    delay_ms(10);

    *r = read_left_color_avg(1);

    left_select_green();

    delay_ms(10);

    *g = read_left_color_avg(1);

    left_select_blue();

    delay_ms(10);

    *b = read_left_color_avg(1);
}

// Read right RGB
void read_right_rgb(long* r,long* g,long* b) {

    right_select_red();

    delay_ms(10);

    *r = read_right_color_avg(1);

    right_select_green();

    delay_ms(10);

    *g = read_right_color_avg(1);

    right_select_blue();

    delay_ms(10);
}

```



```

        *b = read_right_color_avg(1);
    }

// Detect closest color

int detect_color_code(long r,long g,long b) {

    long diff_blue = (r - 330)*(r - 330)
                    + (g - 180)*(g - 180)
                    + (b - 80)*(b - 80);

    long diff_yellow = (r - 30)*(r - 30)
                    + (g - 50)*(g - 50)
                    + (b - 100)*(b - 100);

    long diff_black = (r - 550)*(r - 550)
                    + (g - 600)*(g - 600)
                    + (b - 480)*(b - 480);

    if (diff_black <= diff_blue && diff_black <= diff_yellow) {

        return COLOR_BLACK;

    }

    if (diff_blue <= diff_yellow && diff_blue <= diff_black) {

        return COLOR_BLUE;

    }

    return COLOR_YELLOW;

}

// Convert color code to text

const char* color_name(int color_code) {

    if (color_code == COLOR_BLACK) {

        return "BLACK";

    }

    if (color_code == COLOR_BLUE) {

        return "BLUE";

    }

```

```

    }

    return "YELLOW";
}

// Print RGB + detected colors
void print_color_values_once() {
    long left_r, left_g, left_b;
    long right_r, right_g, right_b;

    read_left_rgb(&left_r, &left_g, &left_b);
    read_right_rgb(&right_r, &right_g, &right_b);

    int left_color = detect_color_code(left_r, left_g, left_b);
    int right_color = detect_color_code(right_r, right_g, right_b);

    uart_send_string("left COLOR: ");
    uart_send_string(color_name(left_color));

    uart_send_string(" R=");
    uart_send_number(left_r);

    uart_send_string(" G=");
    uart_send_number(left_g);

    uart_send_string(" B=");
    uart_send_number(left_b);

    uart_send_string(" right COLOR: ");
    uart_send_string(color_name(right_color));

    uart_send_string(" R=");

```

```

    uart_send_number(right_r);

    uart_send_string(" G=");
    uart_send_number(right_g);

    uart_send_string(" B=");
    uart_send_number(right_b);

    uart_send_string("\r\n");
}

// Simple pseudo random
unsigned long simple_random() {
    random_state = random_state * 1103515245 + 12345;
    return random_state;
}

// Random movement
void random_small_turn() {
    unsigned long r = simple_random() % 3;
    if (r == 0) {
        turn_left_90();
    }
    else if (r == 1) {
        turn_right_90();
    }
    else {
        move_forward();
        delay_ms(150);
    }
}

```

```

// Handle QTI detection

void handle_qti_event() {

    int left_black = left_qti_black();

    int right_black = right_qti_black();

    if (left_black && right_black) {

        uart_send_string("BOTH QTI BLACK: 180\r\n");

        turn_180();

        random_small_turn();

    }

    else if (left_black) {

        uart_send_string("left QTI BLACK: right 90\r\n");

        turn_right_90();

        random_small_turn();

    }

    else if (right_black) {

        uart_send_string("right QTI BLACK: left 90\r\n");

        turn_left_90();

        random_small_turn();

    }

}

// Main program

int main() {

    pins_init();

    timer1_pwm_init();

    uart_init();

    adc_init();

    stop_motors();

    delay_ms(100);

```

```
uart_send_string("START\r\n");

move_forward();

delay_ms(300);

uart_send_string("INITIAL RANDOM TURN\r\n");

random_small_turn();

// main loop
while (1) {

    int left_black = left_qti_black();

    int right_black = right_qti_black();

    // edge detected
    if (left_black || right_black) {

        uart_send_string("BLACK DETECTED: TURN 180\r\n");

        stop_motors();

        delay_ms(100);

        move_backward();

        delay_ms(150);

        turn_180();

        move_forward();

        delay_ms(300);

    }

    else {

        move_forward();

    }

}
```

```
        delay_ms(5);  
    }  
    return 0;  
}
```